Interprocess Communication & Process Synchronization

Outline

- Inter-process communication (IPC)
- Race Conditions, Critical Section, Mutual Exclusion
- Process Synchronization

- Solving the Critical Section problems
  - Software approach (Shared lock variable)
  - Compiler Supported Solution (Semaphores)
  - Hardware-Supported Solutions

- Classical IPC Problems
  - The Producer Consumer Problem
  - Bounded Buffer
  - Readers & Writer Problem and Dinning Philosopher Problem

IPC, Race Conditions, Critical Section, Mutual Exclusion

Inter-process communication (IPC)

- Inter-process communication is the **mechanism provided by the operating system that allows processes to communicate with each other**.
- This communication could involve a process letting another process know that some event has occurred or transferring of data from one process to another.
- Processes in a system can be independent or cooperating.
  - Independent process cannot affect or be affected by the execution of another process.
  - Cooperating process can affect or be affected by the execution of another process.
- Cooperating processes need inter process communication mechanisms.

Exercise Give an example of independent processes?

Inter-process communication (IPC)

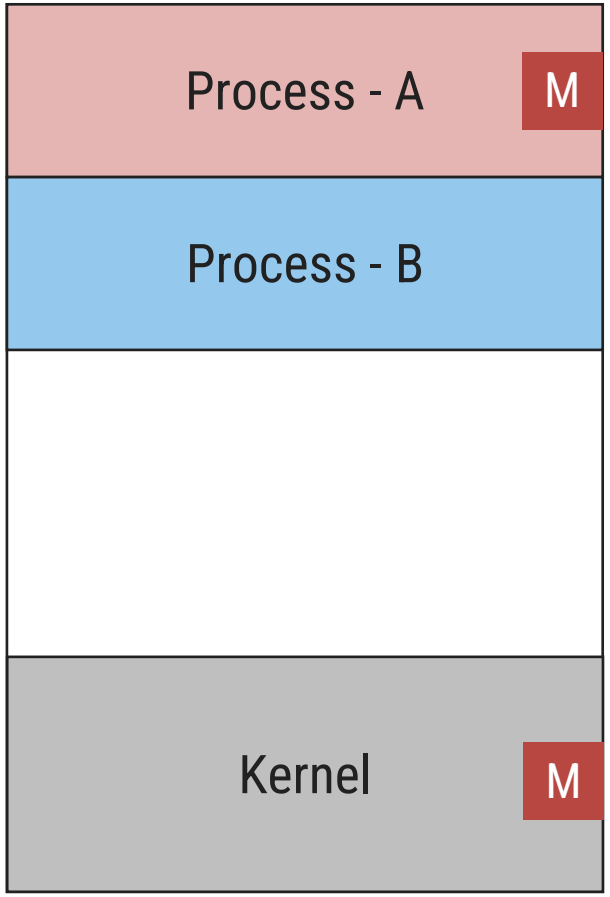
- Reasons of process cooperation
  - **Information sharing**: Several processes may need to access the same data (such as stored in a file.)
  - **Computation speed-up**: A task can often be run faster if it is broken into subtasks and distributed among different processes.
  - **Modularity**: It may be easier to organize a complex task into separate subtasks, then have different processes or threads running each subtask.
  - **Convenience**: An individual user can run several programs at the same time, to perform some task.

- Issues of process cooperation
  - Data corruption, deadlocks, increased complexity
  - Requires processes to synchronize their processing

Models for Inter-process communication (IPC)

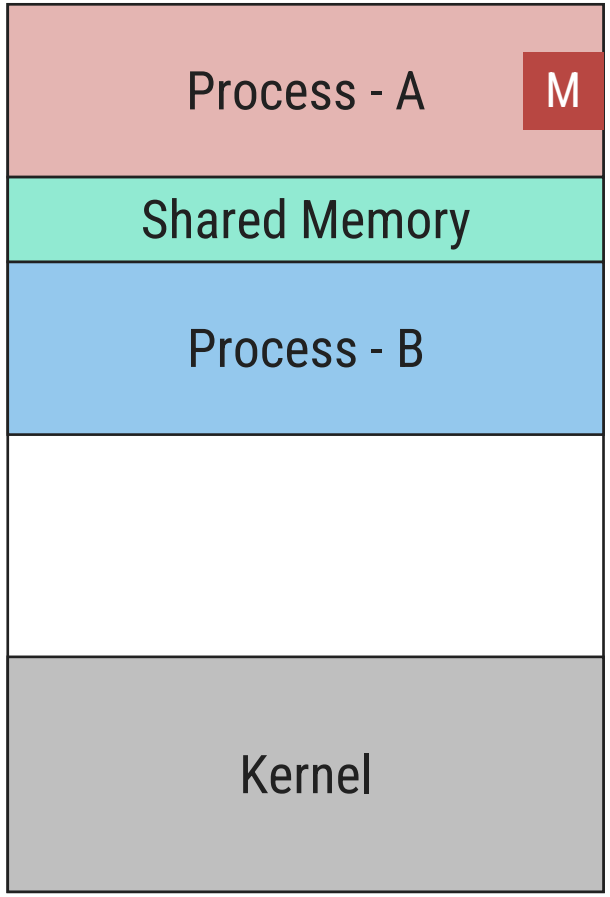
Message Passing

- Process A send the message to Kernel and then Kernel send that message to Process B



Shared Memory

- Process A put the message into Shared Memory and then Process B read that message from Shared Memory

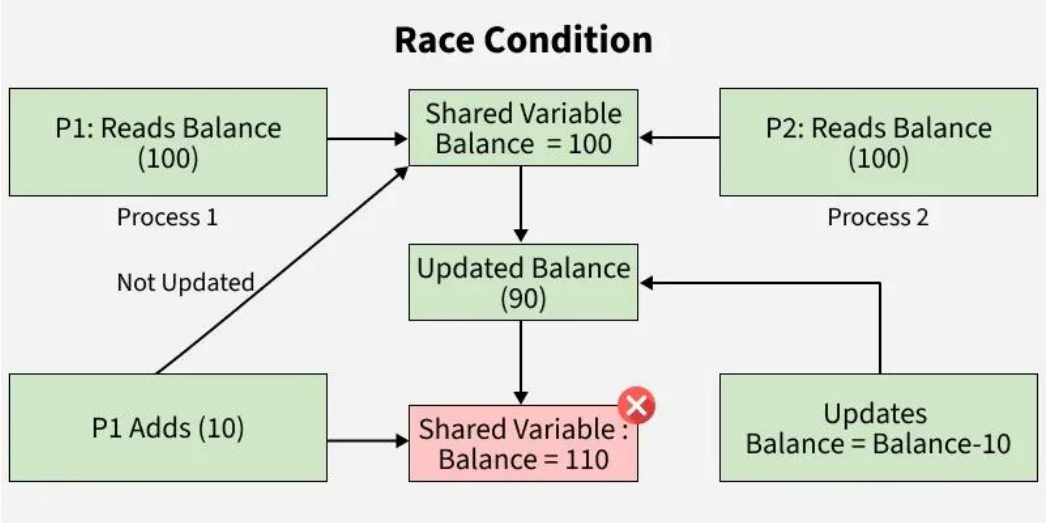


Race Condition

- The situation where **several processes access and manipulate shared data concurrently**. The **final value of the shared data depends upon which process finishes last**.
- A **race condition** is an **undesirable situation** that **occurs when a system attempts to perform two or more operations at the same time**. Improper coordination leads to incorrect or unpredictable results.

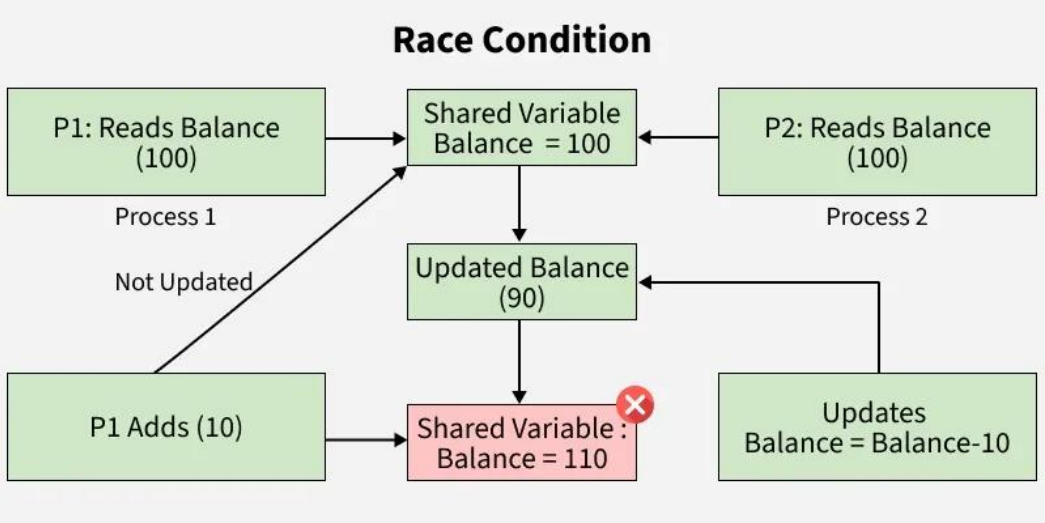
- But, because of the nature of the device or system, the **operations must be done in the proper sequence** to be done correctly.

- **Example: Two Processes Updating a Shared Variable**
  - P1 wants to add 10 to balance.
  - P2 wants to subtract 10 from balance.



Race Condition

- A race condition occurs when two or more processes or threads access and modify the same data at the same time, and the final result depends on the order in which they run. Improper coordination can lead to incorrect or unpredictable results.
- **Example: Two Processes Updating a Shared Variable**
  - P1 wants to add 10 to the balance.
  - P2 wants to subtract 10 from the balance.



- **Updating Shared Variables**:
  - P1 reads balance = 100 and prepares to add 10.
  - Before P1 updates the balance with the new value (110), it is interrupted by the process P2.
  - P2, unaware of P1's action (of adding 10), reads the balance as 100 (incorrect) and prepares to subtract 10.
  - After subtracting, P2 updates the balance to 90 and then P1 resumes and writes the balance as 110 which is incorrect now.
- To prevent race conditions, **concurrent processes must be synchronized**.

Race Condition: Example-2

- Process A
  - next_free_slot = in
  - Write file name at slot (7)
  - next_free_slot += 1
  - in = next_free_slot (8)

Context Switch

- Process B
  - next_free_slot = in
  - Write file name at slot (8)
  - next_free_slot += 1
  - in = next_free_slot (9)

- Process A
  - next_free_slot = in (7)

Context Switch

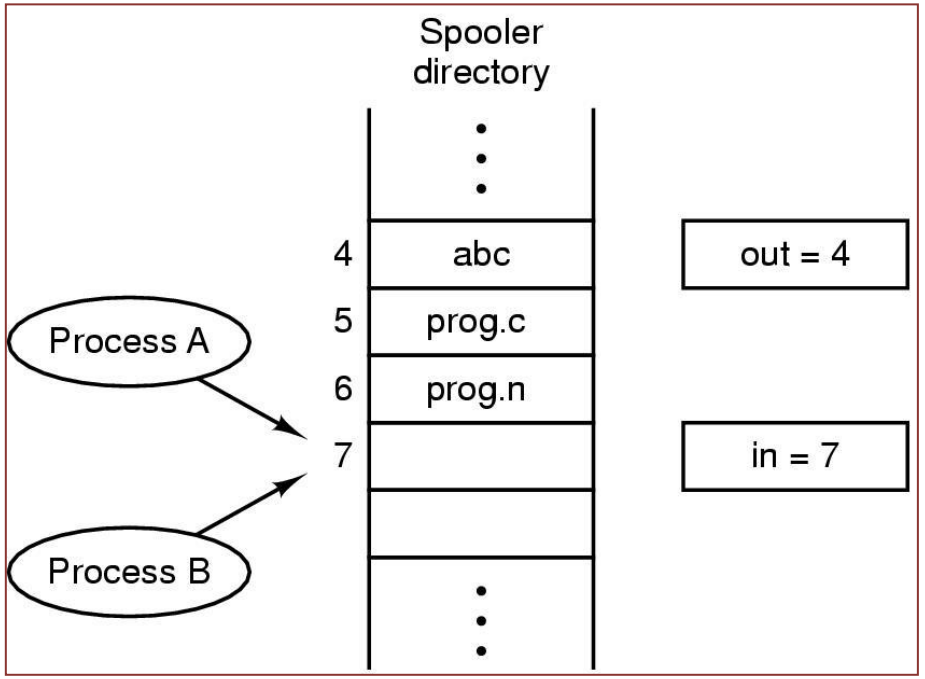
- Process B
  - next_free_slot = in (7)
  - Write file name at slot (7)
  - next_free_slot += 1
  - in = next_free_slot (8)

Context Switch

- Process A
  - Write file name at slot (7)
  - next_free_slot += 1
  - in = next_free_slot (8)

Print spooler directory example :

Two processes want to access shared memory at the same time.



Example of Race Condition

- Process A

  next_free_slot = in

  Write file name at slot (7)

  next_free_slot += 1

  in = next_free_slot (8)

Context Switch

- Process B

  next_free_slot = in

  Write file name at slot (8)

  next_free_slot += 1

  in = next_free_slot (9)

- Process A

  next_free_slot = in (7)

  Context Switch

- Process B

  nex

  ot (7)

  1

  ree_slot (8)

  Context Switch

- Process A

  Write file name at slot (7)

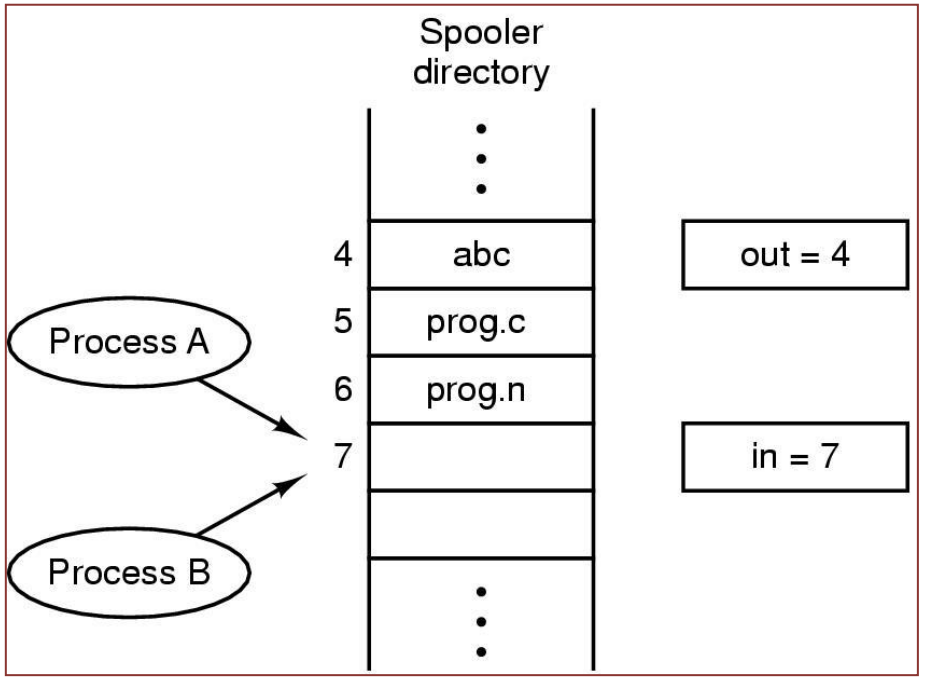
  next_free_slot += 1

  in = next_free_slot (8)

Race Condition

Print spooler directory example :

Two processes want to access shared memory at the same time.

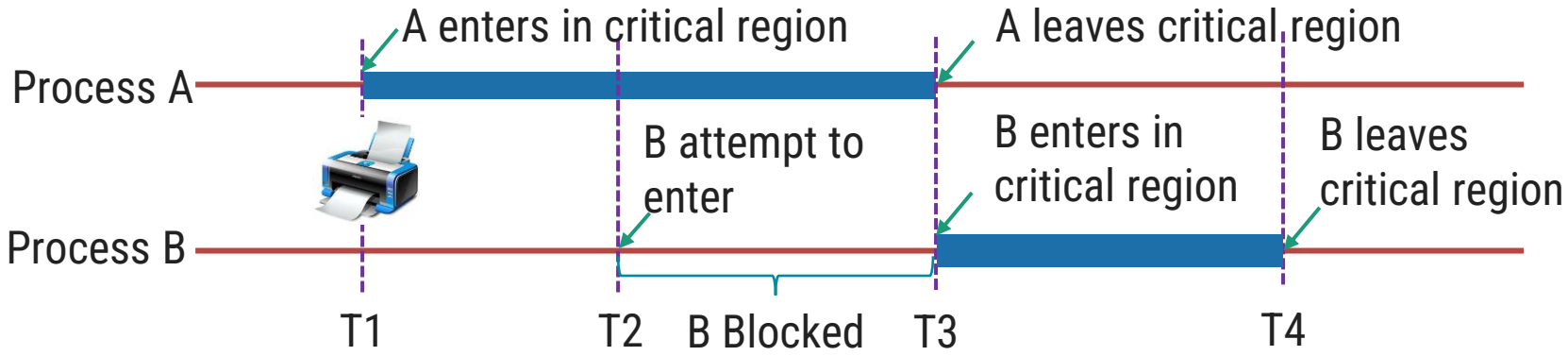


Mutual Exclusion

Mutual Exclusion: Way of making sure that if one process is using a shared variable or file; the other process will be excluded (stopped) from doing the same thing.

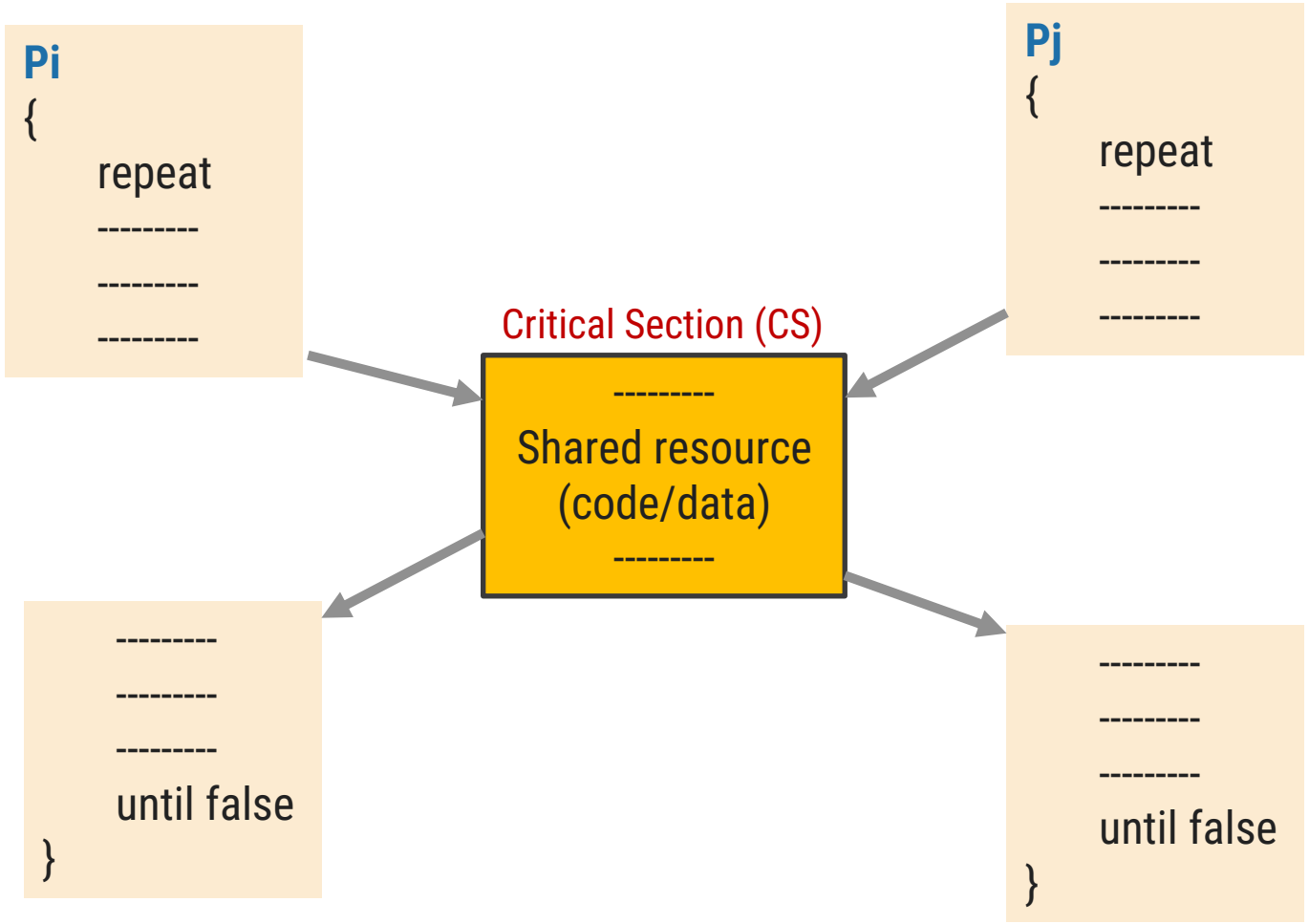
Critical Section

Critical Section: The **part of program where the shared resource is accessed** is called critical section or critical region.



Critical Section Problem

- **Critical Section**: The **part of the program where the shared resource is accessed is called the** critical section or critical region.



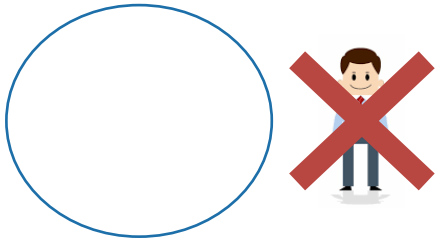
Note: The resource is mutually exclusively shared among processes.

Note:
- There should not be a deadlock.
- No starvation/alternate sharing

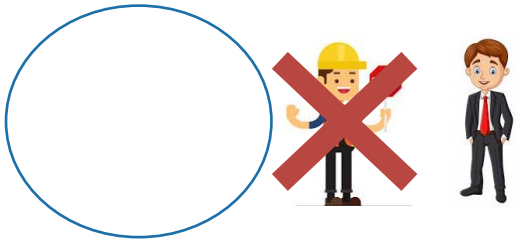
Solving Critical-Section Problem

- Any good solution to the problem must satisfy following four conditions:
- **Mutual Exclusion**
  - No two processes may be simultaneously inside the same critical section.

- **Bounded Waiting** (No Deadlock)
  - No process should have to wait forever to enter a critical section.



- **Progress** (No Alternative sharing/starvation)
  - No process running outside its critical region may block other processes.



- **Arbitrary Speed**
  - No assumption can be made about the relative speed of different processes (though all processes have a non-zero speed).

Process Synchronization

Process Synchronization

- Process synchronization in operating systems is a mechanism that manages the concurrent execution of multiple processes sharing resources to ensure data consistency and prevent race conditions.
- It coordinates process access to shared data, avoiding conflicts and deadlocks, primarily by enforcing orderly execution through techniques like critical sections, semaphores, and locks.
- **Key Concepts in Process Synchronization:**
  - **Race Condition:** Occurs when multiple processes access and manipulate shared data concurrently, causing the outcome to depend on the precise order of execution.
  - **Critical Section:** A segment of code where shared resources are accessed. Only one process should be allowed in this section at a time.
  - **Cooperating Processes:** Processes that can affect or be affected by other processes, requiring synchronization to maintain data integrity.
- **Primary Goals:**
  - **Consistency:** Ensuring data remains accurate when multiple processes are involved.
  - **Orderly Execution:** Coordinating the sequence of process execution.
  - **Resource Management:** Preventing conflicts where two processes try to access the same resource simultaneously.

Process Synchronization: Problems

- **Critical Section**
- Producer Consumer
- Bounded Buffer
- Dining Philosopher
- Reader and Writer

Solution for Critical Section Problem

(Mutual exclusion with busy waiting)

Solution for Critical Section Problem

- Software-based Solution
- Hardware-based Solution
- Compiler Supported Solution

Software-based Solution: Algorithm-1 (Lock Variables)

- **Assume Pi and Pj are two processes.**

```
main()
{
    variable turn : Boolean;
//shared variables between Pi and Pj
        turn = i ;
    par begin
        Pi ;
        Pj ;
    par end
}
```

```
Pi
{
    repeat
    ------
// Entry Section
    while turn ≠ i
    do skip ;
// Exit Section
    turn = j ;
    -----------
    until false
}
```

```
Pj
{
    repeat
    ------
// Entry Section
    while turn ≠ j
    do skip ;
// Exit Section
    turn = i ;
    -----------
    until false
}
```

Critical Section (CS)

Is this Solution Correct?

Note:

- *Starvation or alternate sharing (No progress).*
- *Pi must execute first, Pj cannot enter to CS first.*

Software-based Solution: Algorithm-2

- **Pi and Pj are two processes.**

```
main()
{
    var flag [i, j] : boolean; // Shared variable
    Initialize flag to false
    par begin
        Pi ;
        Pj ;
    par end
}
```

```
Pi
{
    repeat
    ------
// Entry Section
S1: while flag[j] do skip;
    S2: Flag [i] = True;
// Exit Section
Flag[i] = False;
-----------
until false
}
```

```
Pj
{
    repeat
    ------
// Entry Section
S3: while flag[i] do skip;
    S4: Flag [j] = True;
// Exit Section
Flag[j] = False;
-----------
until false
}
```

Critical Section (CS)

Is this Solution Correct?

Note:

- *Mutual exclusion violation.*
- *Both Pi and Pj can access C.S. simultaneously.*

Software-based Solution: Algorithm-3

- **Pi and Pj are two processes.**

```
main()
{
    var flag [i, j] : boolean; // Shared variable
    Initialize flag to false
    par begin
        Pi ;
        Pj ;
    par end
}
```

```
Pi
{
    repeat
    ------
// Entry Section
    S1: Flag [i] = True;
    S2: while flag[j] do skip
// Exit Section
    Flag[i] = False;
    -----------
    until false
}
```

```
Pj
{
    repeat
    ------
// Entry Section
    S3: Flag [j] = True;
    S4: while flag[i] do skip
// Exit Section
    Flag[j] = False;
    -----------
    until false
}
```

Critical Section (CS)

Is this Solution Correct?

Note:

- *Deadlock Present.*
- *Both Pi and Pj can be in a loop. None can go in CS.*

Software-based Solution: Algorithm-4

```
main()
{
    var flag [i, j] : boolean;
    Initialize flag to false
    turn = i ;
    par begin
        Pi ;
        Pj ;
    par end
}
```

```
Pi
{
    repeat
    ------
// Entry Section
S1: Flag [i] = True;
S2: turn= j;
S3: while flag[j] & turn = j do skip;
```

```
Pj
{
    repeat
    ------
// Entry Section
S4: Flag [j] = True;
S5: turn= i;
S6: while flag[i] & turn = i do skip;
```

Critical Section (CS)

```
// Exit Section
Flag[i] = False;
-----------
until false
}
```

```
// Exit Section
Flag[j] = False;
-----------
until false
}
```

Is this Solution Correct?

Note:

- *It is a perfect Solution.*
- *Also known as a Peterson solution.*

Solution for Critical Section Problem

- Software-based Solution
- **Hardware-based Solution**
- Compiler Supported Solution

Solution for Critical Section Problem: Hardware Solution

- **Hardware-based Solution:** The hardware solutions use special types of hardware-supported atomic instructions.
- There are two atomic instructions that are used for a hardware solution.

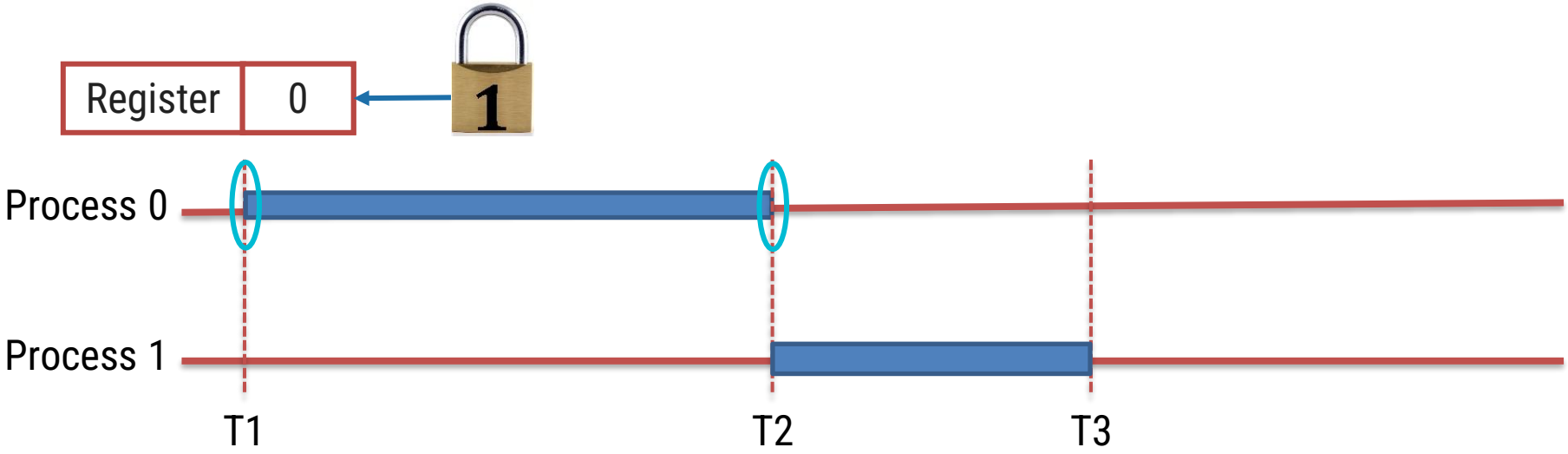
- **Test_and_Set**
  - **Initialize a = false;**

```
function Test_and_Set (var a : boolean) : Boolean
begin
        Test_and_Set = a;
        a =True;  //  a = T;
end
```

  - Test_and_Set is an atomic instruction/function that takes a Boolean argument and returns a Boolean value.
  - Atomic means either execute completely or not at all. It will never execute partially.
  - a is a Boolean variable which is shared among all processes & a is initialized to false.
  - The Test_Set function returns the Boolean value of type a. If a is false, return false; and if a is true, then return true.

TSL (Test and Set Lock) instruction (Hardware approach)

- Test and Set Lock Instruction
  - TSL REGISTER, LOCK
  - It reads the contents of the memory word lock into register RX and then stores a nonzero value at the memory address lock.
  - The operations of reading the word and storing into it are guaranteed to be indivisible—no other processor can access the memory word until the instruction is finished.
  - The CPU executing the TSL instruction locks the memory bus to prohibit other CPUs from accessing memory until it is done.

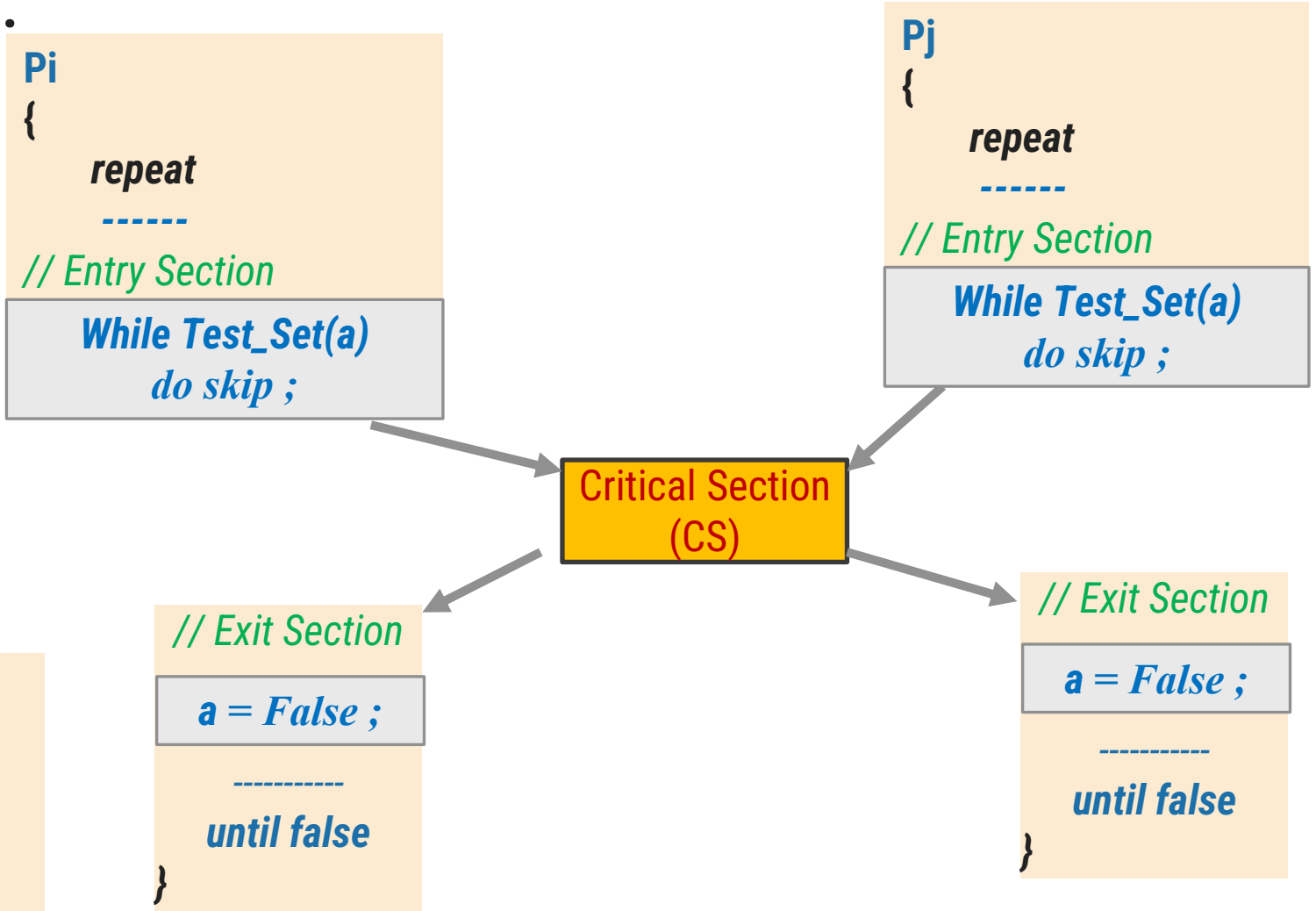


Hardware Solution: Test and Set

- **Assume Pi and Pj are two processes.**

```
main()
{
    variable a : Boolean;
//shared variables between Pi and Pj
        a = false;
    par begin
        Pi ;
        Pj ;
    par end
}
```

```
function Test_Set (var a : boolean) : Boolean
begin
        Test_and_Set = a;
        a =True;  //  a = T;
end
```



Solution for Critical Section Problem: Hardware Solution

- **Hardware-based Solution:** The hardware solutions use special types of hardware-supported atomic instructions.

- There are two atomic instructions that are used for a hardware solution.

  - **Test and Set Lock (TSL)**
    - **Initialize a = false;**

```
function Test_and_Set (var a : boolean) : Boolean
begin
        Test_and_Set = a;
        a =True;  //  a = T;
end
```

  - **Swap/Exchange**

```
Swap(a , b)
{
        t = a;
        a = b;
        b = t;
}
```

Machine Instruction

Exchange instruction (Hardware approach)

- Algorithm

enter_region: (Before entering its critical region, process calls enter_region)

```
MOVE REGISTER, #1      |put 1 in the register
XCHG REGISTER ,LOCK    |swap content of register & lock variable
CMP REGISTER, #0       |was register variable 0?
JNE enter_region       |if it was nonzero, lock was set, so loop
RET                    |return to caller: critical region entered
```

leave_region: (When process wants to leave critical region, process calls leave_region)

```
MOVE LOCK, #0          |store 0 in lock variable
RET                    |return to caller
```

Hardware Solution: Test and Set

- **Assume Pi and Pj are two processes.**

```
main()
{
 variable Lock : Boolean;
//shared variables between Pi and Pj
            Lock = false;
par begin
            Pi ;
            Pj ;
     par end
}
```

```
Swap(a , b)
{
            t = a;
            a = b;
            b = t;
}
```

```
Pi
{
      repeat
       ------
// Entry Section
Key = T;
repeat
    swap(Key, Lock)
until Key = F;
// Exit Section
Lock = False ;
-----------
until false
}
```

```
Pj
{
      repeat
       ------
// Entry Section
Key = T;
repeat
    swap(Key, Lock)
until Key = F;
// Exit Section
Lock = False ;
-----------
until false
}
```

CS

Note: Key is a local variable. Lock is a global shared variable.

Solution for Critical Section Problem

- Software-based Solution
- Hardware-based Solution
- **Compiler Supported Solution (Semaphores)**

Semaphore

Semaphore

- A semaphore is a **variable that provides an abstraction for controlling the access of a shared resource** by multiple processes in a parallel programming environment.

- There are 2 types of semaphores:

  - **Binary semaphores** :-
    - Binary semaphores can take only 2 values (0/1).
    - Binary semaphores have 2 methods associated with it (up, down / lock, unlock).
    - They are used to acquire locks.

  - **Counting semaphores** :-
    - Counting semaphore can have possible values more than two.

Semaphore

- We want functions insert _item and remove_item such that the following hold:
  - **Mutually exclusive access to buffer**: At any time only one process should be executing (either insert_item or remove_item).
  - **No buffer overflow**: A process executes insert_item only when the buffer is not full (i.e., the process is blocked if the buffer is full).
  - **No buffer underflow**: A process executes remove_item only when the buffer is not empty (i.e., the process is blocked if the buffer is empty).
  - **No busy waiting**.
  - **No producer starvation**: A process does not wait forever at insert_item() provided the buffer repeatedly becomes full.
  - **No consumer starvation**: A process does not wait forever at remove_item() provided the buffer repeatedly becomes empty.

Operations on Semaphore

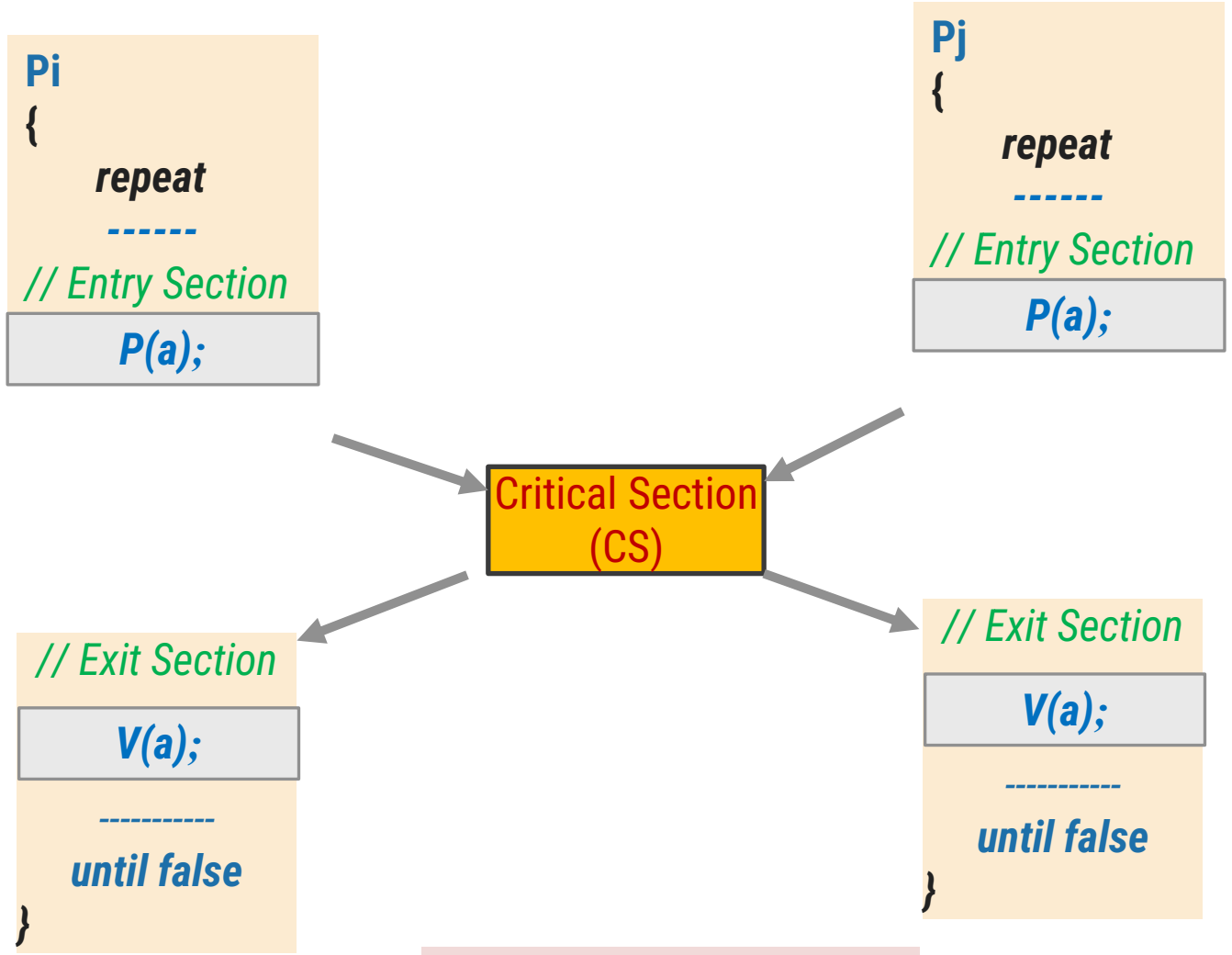
- Wait(): a process performs a wait operation to **tell the semaphore that it wants exclusive access to the shared resource**.
  - If the semaphore is empty, then the semaphore enters the full state and allows the process to continue its execution immediately.
  - If the semaphore is full, then the semaphore suspends the process (and remembers that the process is suspended).

- Signal(): a process performs a signal operation to **inform the semaphore that it is finished using the shared resource**.
  - If there are processes suspended on the semaphore, the semaphore wakes one of the up.
  - If there are no processes suspended on the semaphore, the semaphore goes into the empty state.

Compiler Supported Solution: Semaphores

- **Assume Pi and Pj are two processes.**

```
main()
{
    variable a : Boolean Semaphore;
        initialize a to 1.
    par begin
        Pi ;
        Pj ;
    par end
}
```



Is this Solution Correct?

Note:

- *It is a perfect and simple solution.*
- *It is correct for any number of processes.*

Critical Section Solutions: Mutual Exclusion with Busy Waiting

- Peterson's Solution

- The TSL/XCHG instruction
  - Both Peterson's solution and the solutions using TSL or XCHG are correct.
  - Limitations:
    - Busy Waiting: this approach wastes CPU time
    - Priority Inversion Problem: a low-priority process blocks a higher-priority one

Process Synchronization: Problems

- **Critical Section**
- **Producer Consumer**
- Bounded Buffer
- Dining Philosopher
- Reader and Writer

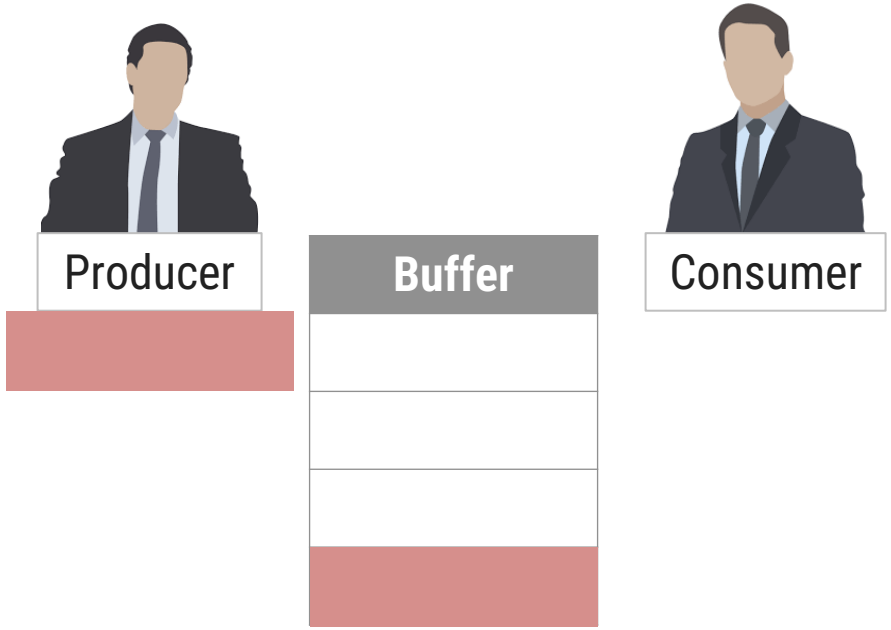
Busy Waiting (Sleep and Wakeup)

- Peterson's solution and solution using TSL and XCHG have the limitation of requiring **busy waiting**.
  - when a processes **wants to enter** in its critical section, it **checks** to see if the entry is allowed.
  - If it is **not allowed**, the process **goes into a loop and waits** (i.e., **start busy waiting**) until it is allowed to enter.
  - This approach **waste CPU-time**.
- But we have interprocess communication primitives (the **pair of sleep & wakeup**).
- **Sleep**: It is a system call that **causes the caller to be blocked** (**suspended**) until some other process wakes it up.
- **Wakeup**: It is a system call that **wakes up** the process.
- Both 'sleep' and 'wakeup' system calls have one parameter that represents a memory address used to match up 'sleeps' and 'wakeups'.

The Producer Consumer Problem

Producer Consumer Problem: Multiple Buffer (Bounded Buffer)

- It is **multi-process synchronization** problem.
- It is also known as **bounded buffer problem**.



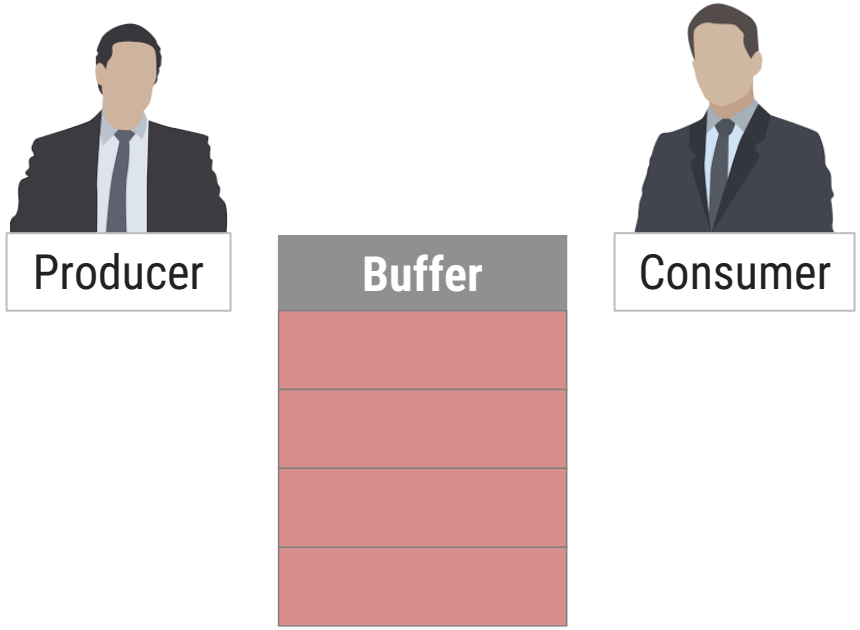
- This problem describes two processes producer and consumer, who share common, fixed size buffer.

- Producer process
  - Produce some information and put it into buffer

- Consumer process
  - Consume this information (remove it from the buffer)

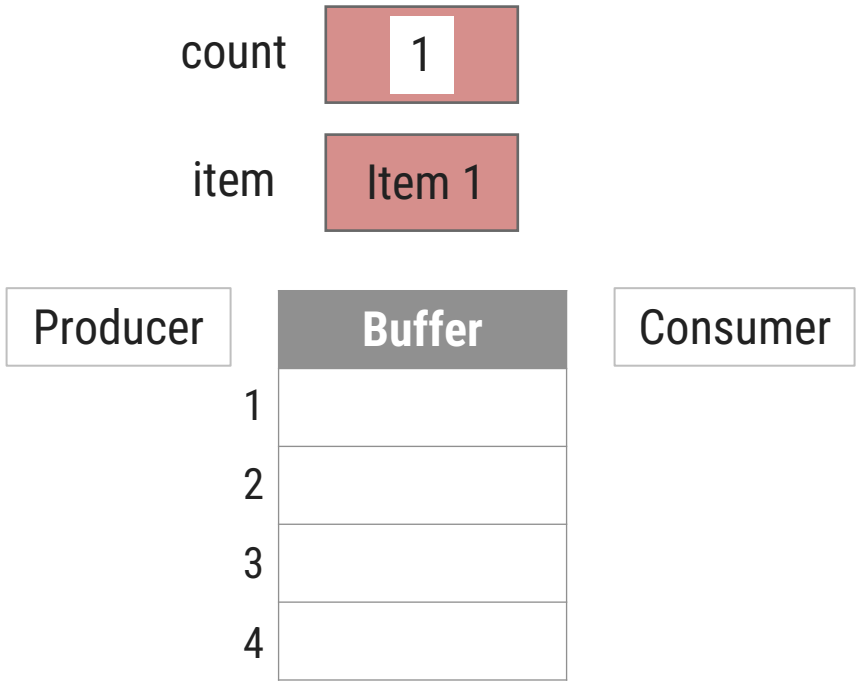
What Producer Consumer (Bounded Buffer) problem is?

- Buffer is **empty**
  - **Producer** want to **produce** √
  - **Consumer** want to **consume** X
- Buffer is **full**
  - **Producer** want to **produce** X
  - **Consumer** want to **consume** √
- Buffer is **partial filled**
  - **Producer** want to **produce** √
  - **Consumer** want to **consume** √



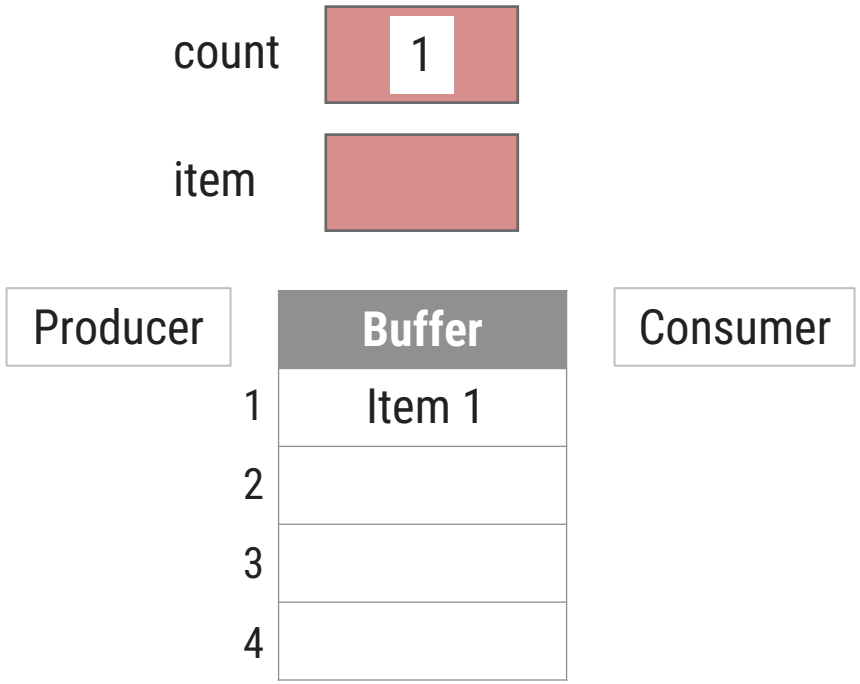
Producer Consumer problem using Sleep & Wakeup

```
#define N 4
int count=0;
void producer (void)
{       int item;
        while (true)
        {       item=produce_item();
                if (count==N) sleep();
                insert_item(item);
                count=count+1;
                if(count==1) wakeup(consumer);
        }
}
```



Producer Consumer (Bounded Buffer) problem using Sleep & Wakeup

```
void consumer (void)
{       int item;
        while (true)
        {       if (count==0) sleep();
                item=remove_item();
                count=count-1;
                if(count==N-1)
                        wakeup(producer);
                consume_item(item);
        }
}
```



Problem in Sleep & Wakeup

- Problem with this solution is that it **contains a race condition that can lead to a deadlock**. (How???)

Producer

```
#define N 4
int count=0;
void producer (void)
{       int item;
        while (true)
        {       item=produce_item();
                if (count==N) sleep();
                insert_item(item);
                count=count+1;
                if(count==1) wakeup(consumer);
        }
}
```

Wakeup Signal

count: 1

item: Item 1

	Buffer
1	
2	Item 2
3	Item 3
4	Item 4

Consumer

```
void consumer (void)
{       int item;
        while (true)
        {       if (count==0)    sleep();
                item=remove_item();
                count=count-1;
                if(count==N-1)
                        wakeup(producer);
                consume_item(item);
        }
}
```

Context Switching

Problem in Sleep & Wakeup

- The **consumer** has just **read the variable count**, noticed **it's zero** and is just about to move inside the if block.
- Just **before calling sleep**, the **consumer is suspended** and the **producer is resumed**.

- The **producer creates an item**, **puts it into the buffer**, and **increases count**.
- Because the **buffer was empty prior to the last addition**, the **producer tries to wake up** the consumer.
- Unfortunately the **consumer wasn't yet sleeping**, and the **wakeup call is lost**.

- When the **consumer resumes**, it **goes to sleep** and will **never be awakened again**. This is because the **consumer is only awakened** by the producer when **count is equal to 1**.
- The **producer will loop until the buffer is full**, after which it will also go to sleep.
- Finally, **both the processes will sleep forever**. This solution therefore is unsatisfactory.

Producer Consumer Problem: Single Buffer

- **Assume Producer and Consumer are two processes.**

```
main()
{
    variable a, b : Boolean Semaphore;
        initialize a to 1 & b to 0.
    par begin
        Producer ;
        Consumer ;
    par end
}
```

```
Producer
{
    repeat
    ------
// Entry Section
P(a);
// Exit Section
V(b);
-----------
until false
}
```

```
Consumer
{
    repeat
    ------
// Entry Section
P(b);
// Exit Section
V(a);
-----------
until false
}
```

Is this Solution Correct?

Note:

- *It is a perfect and simple solution.*

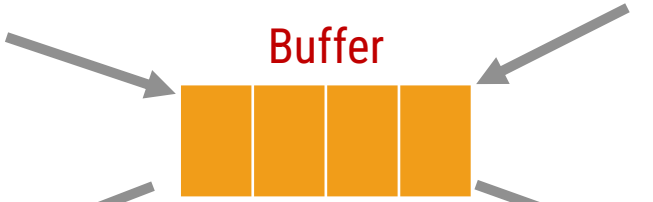
Solution to Producer Consumer Problem: Multiple Buffer

- **Assume Producer and Consumer are two processes.**

```
main()
{
    variable a, b : counting Semaphore;
        initialize a to 4 & b to 0.
    par begin
        Producer ;
        Consumer ;
    par end
}
```

```
Producer
{
    repeat
    ------
// Entry Section
P(a);
```

```
Consumer
{
    repeat
    ------
// Entry Section
P(b);
```




```
// Exit Section
V(b);
-----------
until false
}
```

```
// Exit Section
V(a);
-----------
until false
}
```

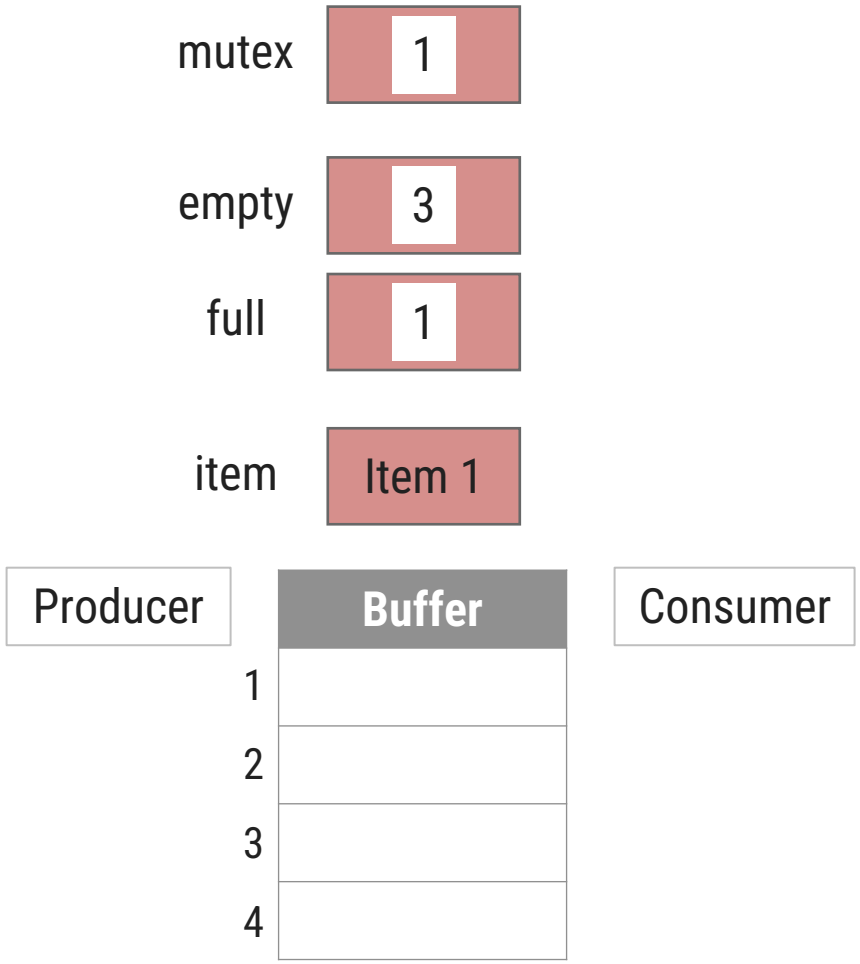
Is this Solution Correct?

Note:

- *It is a perfect and simple solution.*

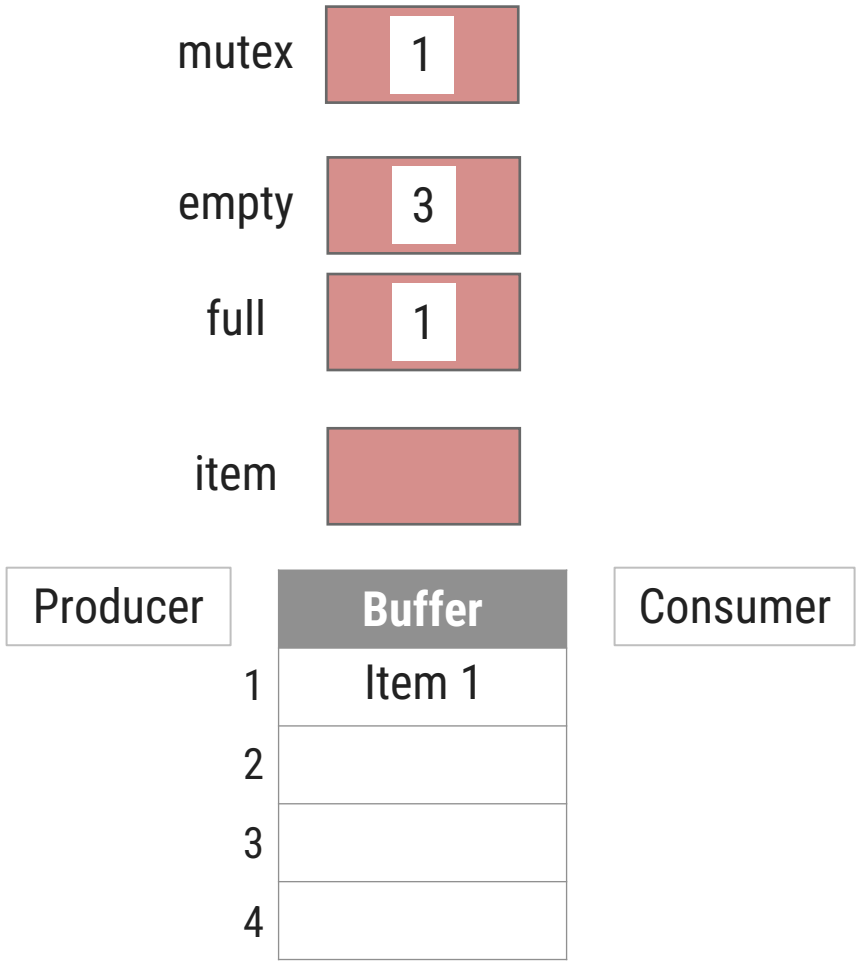
Another Solution to Producer Consumer problem using Semaphore

```
#define N 4
typedef int semaphore;
semaphore mutex=1;
semaphore empty=N;
semaphore full=0;
void producer (void)
{           int item;
            while (true)
            {           item=produce_item();
                        down(&empty);
                        down(&mutex);
                        insert_item(item);
                        up(&mutex);
                        up(&full);          }
}
```



Another Solution to Producer Consumer problem using Semaphore

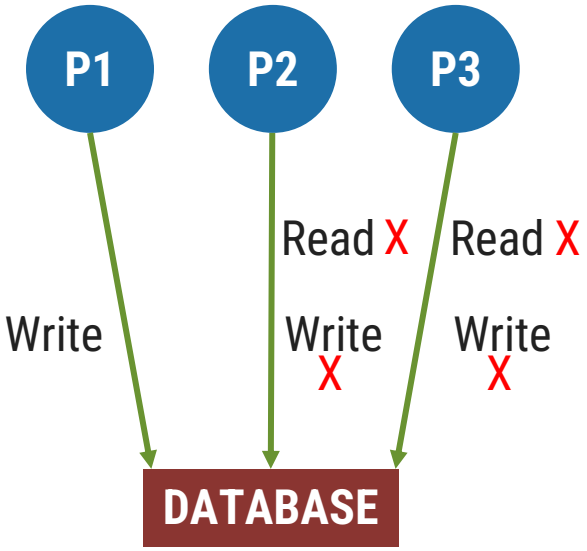
```
void consumer (void)
{       int item;
        while (true)
        {
                down(&full);
                down(&mutex);
                item=remove_item(item);
                up(&mutex);
                up(&empty);
                consume_item(item);
        }
}
```

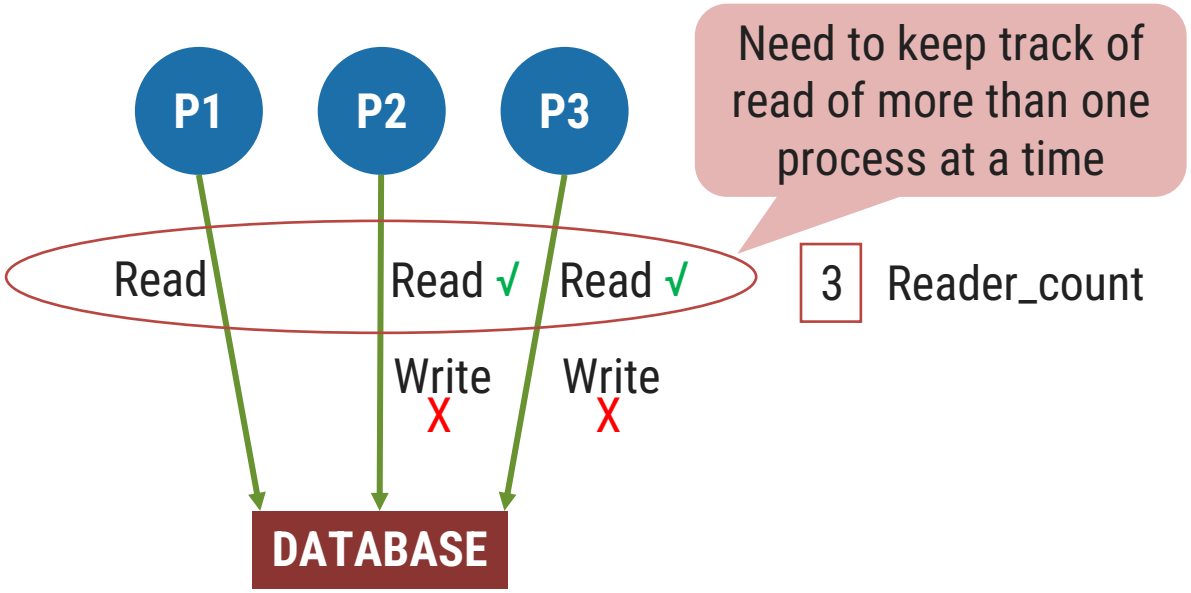


Classical IPC Problems
Readers & Writer Problem

Readers & Writer Problem

- In the readers and writers problem, **many competing processes are wishing to perform reading and writing operations** in a database.





- It is **acceptable to have multiple processes reading the database at the same time**, but **if one process is updating (writing) the database, no other processes may have access to the database**, not even readers.

Solution to Readers & Writer Problem

```
Counting semaphore: mutex=1,  //control access to reader count
Counting semaphore db=1;   //control access to database
int reader_count=0;        //number of processes reading database
```

```
void Writer (void)
{   while (true) {
    create_data();        //create data to enter into DB (non-critical)
    P(db);                //gain access to DB
    write_db();           //write information to DB
    V(db); }              //release exclusive access to DB
}
```

```
void Reader (void)
{     while (true) {
        P(mutex);                        //gain access to reader count
        reader_count=reader_count+1;     //increment reader counter
        if(reader_count==1)              //if this is first process to read DB
            P(db)            //prevent writer process to access DB
        V(mutex)         //allow other process to access reader_count
      read_database();
      P(mutex);                          //gain access to reader count
      reader_count=reader_count-1;       //decrement reader counter
      if(reader_count==0)                //if this is last process to read DB
            V(db)        //leave the control of DB, allow writer process
        V(mutex)         //allow other process to access reader_count
        use_read_data(); }   //use data read from DB (non-critical)
}
```

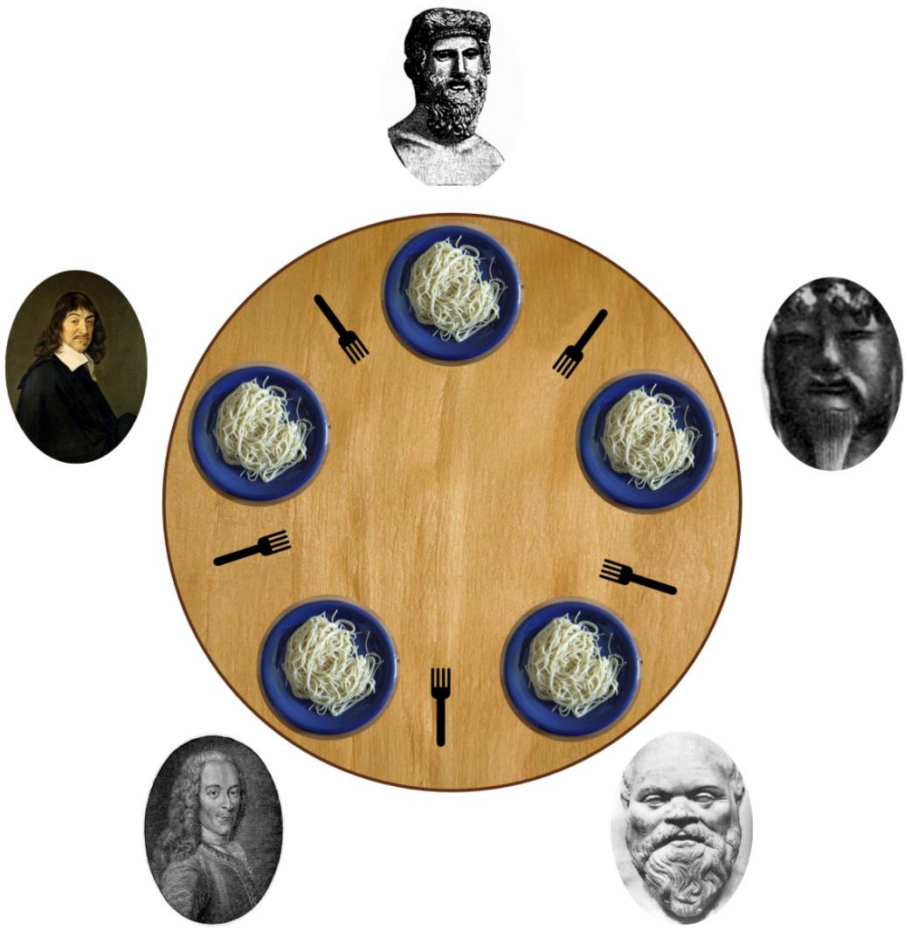
Classical IPC Problems
Dinning Philosopher Problem

Dinning Philosopher Problem

- In this problem **5 philosophers sitting at a round table doing 2 things eating and thinking**.
- While **eating** they are **not thinking** and while **thinking** they are **not eating**.

- **Each philosopher has plates** that is total of **5 plates**.
- And there is a **fork place between each pair** of adjacent philosophers that is **total of 5 forks**.

- **Each philosopher needs 2 forks to eat** and each **philosopher can only use the forks on his immediate left and immediate right**.



Solution to Dinning Philosopher Problem

- **Assume Pi is a one philosopher Process.**

Chopstick

0	0	0	0	0

```
Main Program
{
    variable array chopstick [0:4]: Boolean Semaphore;
            Initialize chopstick to 1
  par begin
            P0 ;        // Philosopher 1
            P1;         // Philosopher 2
            P2;         // Philosopher 3
            P3;         // Philosopher 4
            P4          // Philosopher 5
par end
}
```

```
Pi
{
    repeat
    ------
// Entry Section
S1: P (chopstick[i]);
S2: P (chopstick[(i + 1) mod 5]);

Eating

// Exit Section
S3: V (chopstick[i]);
S4: V (chopstick[(i + 1) mod 5]);

-----------
---Think--
until false
}
```

Is this Solution Correct?

Note:

- *Here, deadlock will occur due to picking up one chopstick by each process.*

Solution to Dinning Philosopher Problem

- **Assume Pi is a one philosopher Process.**

Chopstick	0	0	0	0	0

```
Main Program
{
    variable array chopstick [0:4]: Boolean Semaphore;
        Initialize chopstick to 1
  par begin
        P0 ;        // Philosopher 1
        P1;         // Philosopher 2
        P2;         // Philosopher 3
        P3;         // Philosopher 4
        P4          // Philosopher 5
par end
}
```

```
Pi
{
    repeat
    ------
// Entry Section
S1: P (chopstick[i]);
S2: P (chopstick[(i + 1) mod 5]);

Eating

// Exit Section
S3: V (chopstick[i]);
S4: V (chopstick[(i + 1) mod 5]);
----------
----------
---Think--
until false
}
```

```
P4
{
    repeat
    ------
// Entry Section
S5: P (chopstick[(i + 1) mod 5]);
S6: P (chopstick[i]);

Eating

// Exit Section
S7: V (chopstick[(i + 1) mod 5]);
S8: V (chopstick[i]);
----------
----------
---Think--
until false
}
```

Note:

- *Change the entry and exit code for one process (P4) as given.*
- *Then the solution is perfect.*

Another Solution to Dinning Philosopher Problem

```
#define N 5                    //no. of philosophers
#define LEFT (i+N-1)%5         //no. of i's left neighbor
#define RIGHT (i+1)%5          //no. of i's right neighbor
#define THINKING 0             //Philosopher is thinking
#define HUNGRY 1               //Philosopher is trying to get forks
#define EATING 2               //Philosopher is eating
typedef int semaphore;         //semaphore is special kind of int
int state[N];                  //array to keep track of everyone's state
semaphore mutex=1;             //mutual exclusion for critical region
semaphore s[N];                //one semaphore per philosopher
```

Another Solution to Dinning Philosopher Problem

```
void philosopher (int i)   //i: philosopher no, from 0 to N-1
{
    while (true)
        {
            think();          //philosopher is thinking
            take_forks(i);    //acquire two forks or block
            eat();            //eating spaghetti
            put_forks(i);     //put both forks back on table
        }
}
```

```
void take_forks (int i)          //i: philosopher no, from 0 to N-1
{           down(&mutex);        //enter critical region
            state[i]=HUNGRY;     //record fact that philosopher i is hungry
            test(i);             //try to acquire 2 forks
            up(&mutex);          //exit critical region
            down(&s[i]);         //block if forks were not acquired
}
```

```
void test (i)                                   //i: philosopher no, from 0 to N-1
{ if (state[i]==HUNGRY && state[LEFT]!=EATING &&
state[RIGHT]!=EATING)
            {           state[i]=EATING;
                        up (&s[i]); }
}
```

```
void put_forks (int i)           //i: philosopher no, from 0 to N-1
{           down(&mutex);        //enter critical region
            state[i]=THINKING;   //philosopher has finished eating
            test(LEFT);          //see if left neighbor can now eat
             test(RIGHT);        // see if right neighbor can now eat
            up(&mutex);          // exit critical region
}
```

Monitor

- A higher-level **synchronization primitive**.
- A monitor is a **collection of procedures, variables, and data structures** that are all **grouped together in a special kind of module or package**.
- **Processes may call the procedures** in a monitor whenever they want to, but they **cannot directly access the monitor's internal data structures** from procedures declared outside the monitor.
- Monitors have an important property for achieving mutual exclusion: **only one process can be active in a monitor at any instant**.
- When a process calls a monitor procedure, the **first few instructions of the procedure will check to see if any other process is currently active within the monitor**.
- If so, the **calling process will be suspended** until the other process has left the monitor. If no other process is using the monitor, the calling process may enter.

Mutex

Mutex

- Mutex is the short form for '**Mutual Exclusion Object**'.
- A Mutex and the binary semaphore are essentially the same.
- Both **Mutex and the binary semaphore can take values: 0 or 1**.

```
mutex_lock:
    TSL REGISTER,MUTEX      |copy Mutex to register and set Mutex to 1
    CMP REGISTERS, #0       |was Mutex zero?
    JZE ok                  |if it was zero, Mutex was unlocked, so return
    CALL thread_yield       |Mutex is busy; schedule another thread
    JMP mutex_lock          |try again later
ok:         RET             |return to caller; critical region entered

mutex_unlock:
    MOVE MUTEX,#0           |store a 0 in Mutex
    RET                     |return to caller
```

Thank You